

Съдържание

Увод	1
Глава 1. Обзор на съществуващи решения	3
1.1. Анализ на сходни системи и сравнение между тях	3
1.1.1. The North Face	3
1.1.2. Patagonia	4
1.1.3. Amazon	5
1.1.4. Decathlon	5
1.2. Обобщителна информация	5
1.3. Изводи	7
1.4. Цели и задачи	8
1.4.1. Цели	8
1.4.2. Задачи	8
1.5. Заключение	9
Глава 2. Проект на решението	10
2.1. Архитектурен модел	10
2.2. Функционален модел	12
2.2.1. Сценарии	14
2.3. Проектиране на интерфейса	18
2.4. Организация на данните	21
2.4.1. Entity Relationship Diagram	21
2.4.2. База данни	22
Глава 3. Избор на средства и описание на реализацията	27
3.1. Избор на средства	27
3.1.1. Избор на език	27
3.1.2. Избор на софтуерен фреймуърк	30
3.1.3. Избор на уеб сървър	30
3.1.4. Инфраструктура и разполагане	30
3.2. Описание на реализацията	30
3.2.1. Организация на сървърната част	31
3.2.2. Приложно-програмен интерфейс	32
3.2.3. Контрол на достъпа	33
3.2.4. Работа с базата данни	33
3.2.5. Интеграция с картографски услуги	34
3.2.6. Прогноза за времето	34
3.2.7. Препоръка за екипировка	34
3.2.8. Плащане	35
3.2.9. Проследяване на поръчки	35
3.2.10. Организация на клиентската част	36
Глава 4. Описание на тестването на предложеното решение	36

4.1. Подход към тестването	36
4.2. Тестване по метода на затворената кутия	37
4.3. Тестване по метода на частично отворената кутия	38
4.4. Тестване по метода на отворената кутия	40
4.5. Открити дефекти	41
4.6. Отстраняване на дефектите	42
4.7. Обобщение	44
Глава 5. Бъдеща визия	45
5.1. Технически задачи с непосредствен приоритет	45
5.2. Разширяване на функционалността	45
Заклучение	47
Литература	48

Увод

През последните години онлайн пазаруването се превърна в нормална част от ежедневието. Това важи и за продуктите, свързани със спорт, туризъм и планински преходи. В интернет съществуват много магазини, от които потребителите могат да закупят дрехи, обувки и различно туристическо оборудване. В повечето случаи обаче изборът на продукт се извършва само според неговите характеристики, цена или марка.

При туристическата екипировка има и други фактори, които могат да бъдат важни при избора. Например при планиране на конкретен преход значение имат трудността и дължината на маршрута, мястото, където се намира, както и очакваното време в деня на прехода. Поради тази причина при разработването на настоящия проект беше поставена идеята електронният магазин да бъде комбиниран с информация за туристически маршрути и метеорологични условия.

В дипломната работа е разработена уеб система **TrailBlaze**, която представлява онлайн магазин за туристическа екипировка с допълнителни функции за търсене и разглеждане на туристически маршрути. Потребителят може да разглежда продуктите, да използва различни филтри, да добавя продукти в количка или списък с желани продукти и да извършва поръчка. Системата включва и регистрация и вход на потребители, управление на профил и проследяване на направените поръчки.

Една от основните функционалности на проекта е търсенето на туристически маршрути. Потребителят може да търси маршрут по град, парк или регион. За намерените маршрути се показва информация за тях, като име, трудност и разстояние, както и интерактивна карта. За реализирането на тази част са използвани данни от OpenStreetMap и няколко външни услуги за работа с географска и маршрутна информация.

Към системата е добавена и информация за метеорологичната прогноза. Тя се използва и в друга функционалност на проекта – **AI Gear Advisor**. При избор на маршрут и дата системата събира информация за маршрута и очакваните метеорологични условия. След това тези данни се използват заедно с наличните продукти в магазина, за да бъде предложена подходяща екипировка. За тази функционалност е използван Claude на Anthropic.

Целта на дипломната работа е да се разработи уеб приложение, което да съчетава електронен магазин за туристическа екипировка с функционалности за търсене на маршрути и препоръчване на подходящи продукти.

За постигането на тази цел са поставени следните основни задачи:

- проучване на съществуващи онлайн магазини за туристическа екипировка;
- определяне на необходимите функционалности на системата;
- проектиране на структурата на приложението и базата данни;
- разработване на каталог с продукти и филтри;
- реализиране на количка, wishlist и поръчки;
- реализиране на регистрация и вход на потребители;
- интегриране на онлайн плащане чрез Stripe;
- разработване на търсене и визуализиране на туристически маршрути;
- интегриране на метеорологична информация;
- разработване на AI функционалност за препоръчване на екипировка;
- тестване на разработените функционалности.

За реализацията на проекта е използван **React** за клиентската част и **Node.js с Express** за сървърната част. Данните се съхраняват в PostgreSQL база данни, като достъпът до нея се извършва чрез Prisma. Използвани са още TypeScript, Vite, Tailwind CSS и други библиотеки, необходими за разработването на интерфейса и отделните функционалности.

Системата използва и няколко външни услуги – Stripe за плащания, OpenStreetMap и Overpass API за информация за маршрути, OpenRouteService за маршрутни направления, Open-Meteo за прогнозата за времето и Anthropic за AI препоръките.

В следващите части на дипломната работа са разгледани съществуващи решения, направено е сравнение с разработената система, представени са проектът и архитектурата на TrailBlaze, описана е реализацията на основните функционалности и са представени резултатите от проведеното тестване.

Глава 1. Обзор на съществуващи решения

1.1. Анализ на сходни системи и сравнение между тях

При разработването на TrailBlaze е направено проучване на вече съществуващи електронни магазини, които предлагат туристическо, спортно и планинско

оборудване. Целта на анализа е да се разгледат основните функционалности, които се използват при подобни системи, както и да се установи с какво разработената система се различава от тях.

За сравнение са избрани **The North Face, Patagonia, Amazon и Decathlon**. Първите две платформи са насочени основно към outdoor продукти, докато Amazon и Decathlon предлагат значително по-широк набор от спортни и туристически стоки.

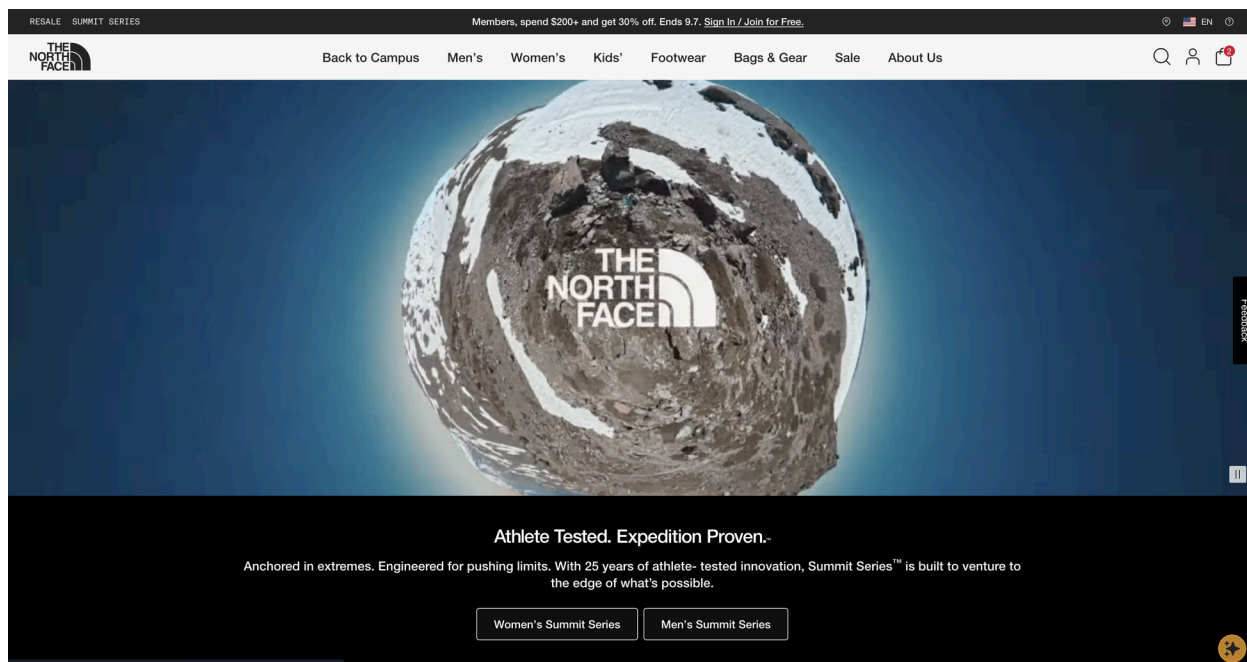
1.1.1. The North Face

The North Face е онлайн платформа на компания, специализирана в производството и продажбата на облекло, обувки и оборудване за outdoor дейности. Сайтът предлага отделни категории за мъже, жени и деца, както и категории за обувки, чанти и оборудване. Потребителят може да разглежда продуктите според тяхното предназначение и да използва допълнителни инструменти за избор на определени видове продукти. Сред тях са например инструменти за избор на раница, яке и обувки.

Системата разполага и със стандартните за електронен магазин функционалности – търсене на продукти, информация за продукта, любими продукти, проследяване на поръчки, информация за доставка и връщане.

The North Face представлява добър пример за специализиран онлайн магазин, тъй като съдържанието и продуктовете категории са насочени към хора, които използват екипировка за различни outdoor дейности.

Основното ограничение, което може да бъде разгледано от гледна точка на настоящия проект, е че потребителят основно започва от продукта. Ако например човек планира конкретен туристически маршрут, той трябва самостоятелно да определи каква екипировка му е необходима и да потърси подходящите продукти.



Фигура 1.1 - Начална страница на The North Face

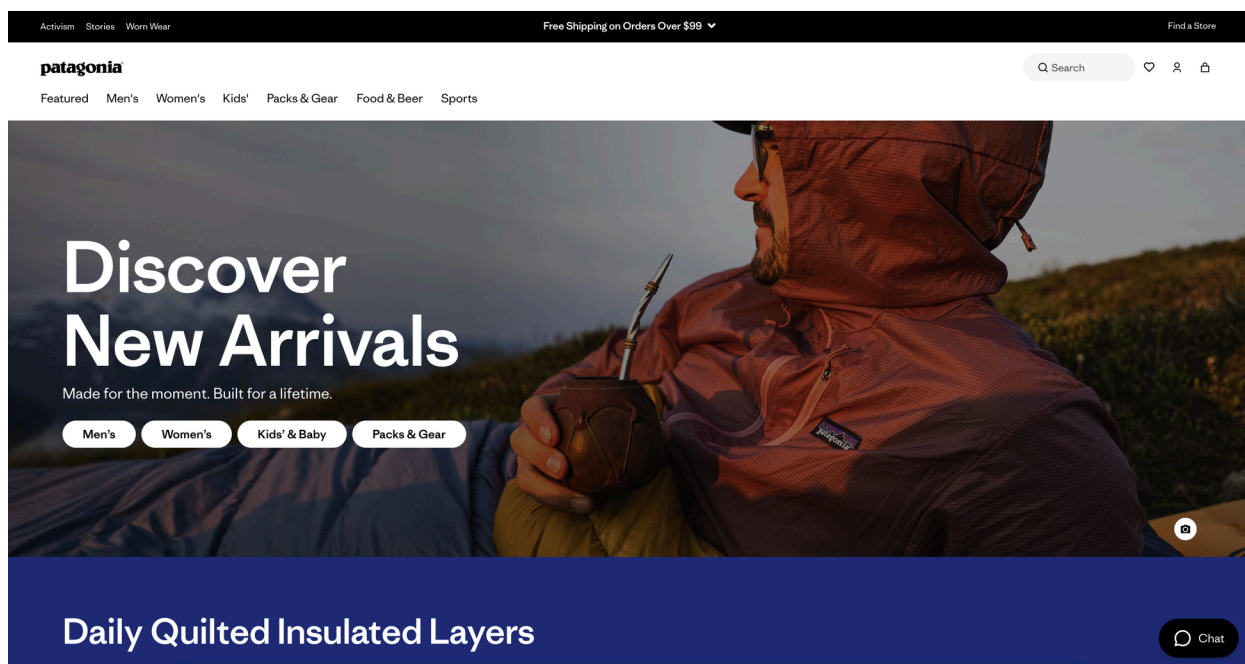
1.1.2. Patagonia

Patagonia е друг пример за специализиран онлайн магазин за outdoor облекло и оборудване. Платформата предлага категории за различни видове облекло и екипировка, като има възможност за филтриране по размер, цена, цвят, характеристики, материал и спорт. Сред поддържаните дейности е и hiking.

При разглеждането на продуктите потребителят получава информация за техните характеристики, оценки и предназначение. Сайтът включва и допълнително съдържание, свързано с използването и поддръжката на екипировката.

Предимството на Patagonia е доброто представяне на продуктите и възможността за сравнително подробно филтриране. Платформата е ориентирана към конкретна аудитория и предоставя голям избор от продукти за различни outdoor дейности.

При TrailBlaze обаче е добавен друг подход. Вместо потребителят сам да преценява кои продукти са подходящи, системата позволява първо да бъде избран конкретен маршрут и дата. След това информацията за маршрута и времето може да бъде използвана от AI Gear Advisor за създаване на препоръка.



Фигура 1.2 - Начална страница на Patagonia

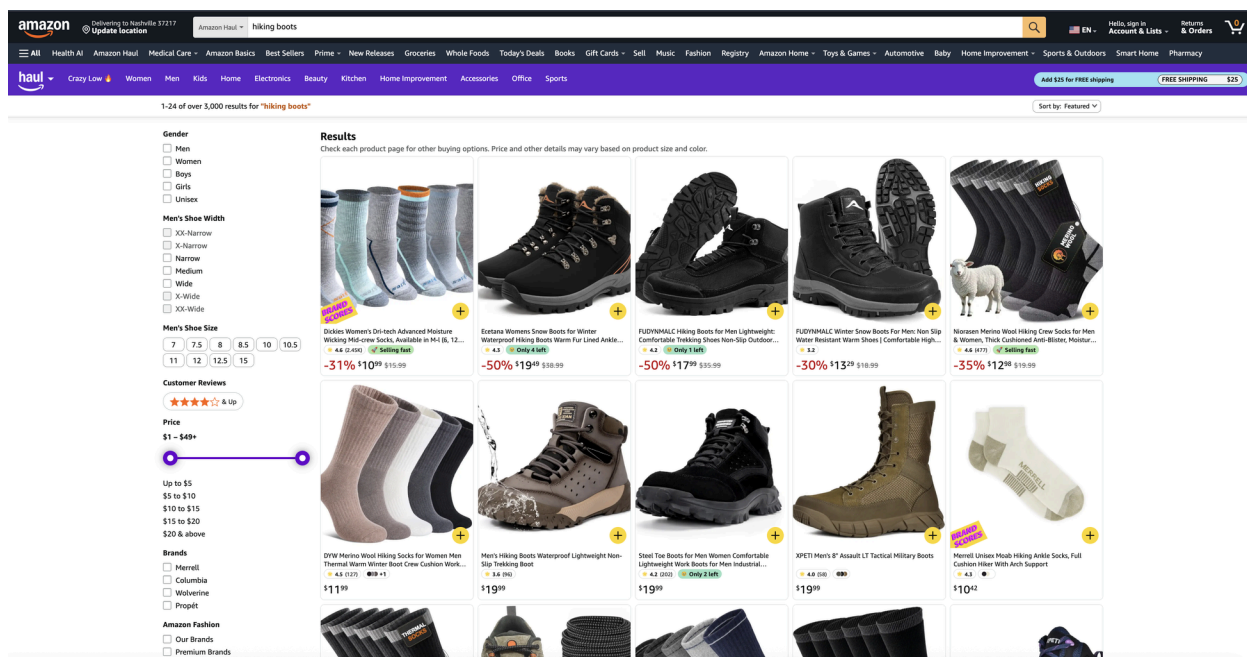
1.1.3. Amazon

Amazon е значително по-голяма и универсална електронна платформа. За разлика от The North Face и Patagonia, тя не е специализирана само в туристическа екипировка. В платформата съществува отделна категория за outdoor recreation, включително camping и hiking, в която се предлагат различни видове облекло, обувки, раници, палатки, туристически аксесоари и друго оборудване.

Едно от основните предимства на Amazon е големият избор от продукти и различни производители. Платформата предоставя и голямо количество информация от потребители – оценки, ревюта и други данни, които могат да подпомогнат избора на продукт.

От друга страна, големият брой продукти може да затрудни избора, когато потребителят няма предварително определена представа какво точно търси. В случая с туристически преход това означава, че потребителят отново трябва сам да прецени какво му е необходимо.

TrailBlaze използва по-тясна специализация. Каталогът е насочен към туристическа екипировка, а функционалността за маршрути позволява изборът на продукт да бъде свързан с конкретна дейност.



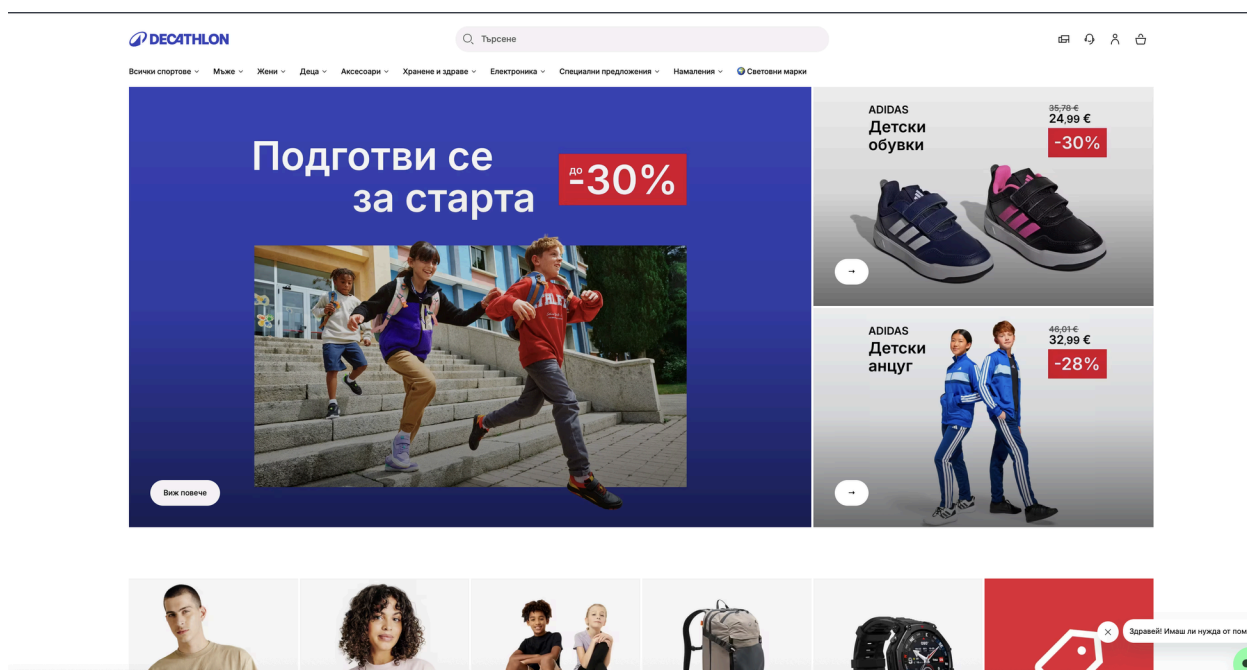
Фигура 1.3 - Резултати от търсене на екипировка в Amazon

1.1.4. Decathlon

Decathlon е пример за голям онлайн магазин, ориентиран към спортни и туристически продукти. Подобно на Amazon, платформата предлага много различни категории и продукти, но е по-силно насочена към спортната и активната дейност.

При такъв тип система основният модел отново е избор на спорт, категория и продукт. Това позволява на потребителя да намери необходимата екипировка, но не обвързва директно избора с конкретен туристически маршрут и актуалните условия по него.

За целите на TrailBlaze този модел е полезен като пример за организиране на голям продуктов каталог, категории и филтриране. В разработената система обаче каталогът е свързан с допълнителна информация за маршрута и прогнозата за времето.



Фигура 1.4 - Начална страница на Decathlon

1.2. Обобщителна информация

От разгледаните решения може да се види, че съвременните онлайн магазини предлагат голяма част от необходимите функции за електронна търговия. Потребителят може да разглежда продукти, да ги филтрира, да ги добавя в количка, да извършва плащане и да следи своите поръчки. При специализираните магазини за туристическа екипировка има и допълнителна информация за предназначението и характеристиките на продуктите.

При разработването на TrailBlaze тези основни функции също са включени. Разликата е, че към електронния магазин са добавени функции, свързани директно с планирането на туристически преход. Потребителят може да потърси маршрут, да го разгледа на карта, да получи информация за неговата трудност и дължина и да провери прогнозата за избрана дата.

Друга особеност на разработената система е AI Gear Advisor. Той използва информацията за маршрута и времето, за да предложи подходящи продукти от наличните в магазина. По този начин потребителят не трябва сам да прави връзката между условията на прехода и необходимата екипировка.

Следващата таблица представя основните разлики между разгледаните решения и TrailBlaze.

Функционалност	The North Face	Patagonia	Amazon	Decathlon	TrailBlaze
Онлайн магазин	✓	✓	✓	✓	✓
Категории и филтри	✓	✓	✓	✓	✓
Продуктова информация	✓	✓	✓	✓	✓
Количка и поръчки	✓	✓	✓	✓	✓
Wishlist	✓	✓	✓	✓	✓
Онлайн плащане	✓	✓	✓	✓	✓
Потребителски профил	✓	✓	✓	✓	✓
Търсене на туристически маршрути	–	–	–	–	✓
Интерактивна карта	–	–	–	–	✓
Информация за времето	–	–	–	–	✓
Избор на дата за преход	–	–	–	–	✓
AI препоръка за екипировка	–	–	–	–	✓
Препоръка според конкретен маршрут	–	–	–	–	✓

От таблицата се вижда, че основните функции на електронната търговия са сходни при всички разгледани системи. Основната разлика при TrailBlaze е добавянето на функции, които свързват продукта с конкретния начин, по който той ще бъде използван.

1.3. Изводи

- След направения анализ може да се направи извод, че съществуващите електронни магазини предоставят добри възможности за търсене и закупуване на туристическа екипировка. Те обаче основно се концентрират върху самите продукти и процеса на покупка.
- При планиране на туристически преход потребителят трябва допълнително да търси информация за маршрута, неговата трудност и очакваното време. След това сам трябва да прецени каква екипировка ще му бъде необходима.
- Това е основната идея, върху която е разработена TrailBlaze. Системата събира тези действия на едно място и позволява информацията за маршрута и метеорологичните условия да бъде използвана при избора на продукти.
- По този начин TrailBlaze не се стреми да замени големите онлайн магазини по отношение на броя продукти, а предлага по-специализиран начин за избор на туристическа екипировка.

1.4. Цел и задачи

1.4.1. Цел

Целта на дипломната работа е да се разработи уеб базирана система за продажба на туристическа екипировка, която освен стандартните функции на онлайн магазин да предоставя възможност за търсене на туристически маршрути и подпомагане на избора на подходящо оборудване.

За постигането на тази цел системата трябва да обедини няколко основни функционални части – продуктов каталог, потребителски профили, количка и поръчки, плащане, търсене на маршрути, информация за времето и AI препоръки.

Крайният потребител следва да може да премине през по-голяма част от подготовката за своя преход в рамките на една система.

1.4.2. Задачи

За реализиране на поставената цел са определени следните задачи:

- Да се проучат съществуващи онлайн магазини за туристическа и спортна екипировка.
- Да се определят основните функционални изисквания към разработваната система.
- Да се проектира структурата на клиентската и сървърната част на приложението.
- Да се проектира база данни за съхраняване на потребители, продукти, колички, wishlist-и, поръчки и адреси за доставка.
- Да се реализира регистрация, вход и управление на потребителски профил.
- Да се разработи продуктов каталог с категории и възможности за филтриране.
- Да се реализират количка и wishlist.
- Да се реализира процес за извършване и проследяване на поръчки.
- Да се интегрира платежна система чрез Stripe.
- Да се разработи търсене на туристически маршрути чрез външни картографски услуги.
- Да се добави интерактивна карта за визуализиране на избрания маршрут.
- Да се интегрира услуга за получаване на метеорологична прогноза.
- Да се разработи AI Gear Advisor за препоръчване на подходяща екипировка.
- Да се свържат AI препоръките с наличните продукти в системата.
- Да се проведе тестване на основните функционалности и да се проверят резултатите от тяхната работа.

1.5. Заключение

В тази глава бяха разгледани няколко съществуващи решения в областта на електронната търговия и туристическата екипировка. Бяха сравнени техните основни функционалности и беше разгледано с какво TrailBlaze се различава от тях.

От направеното сравнение се вижда, че основните функции на онлайн магазина вече са добре познати и се използват в много съществуващи системи. Основната особеност на TrailBlaze е добавянето на туристически маршрути, информация за времето и AI препоръки към стандартния процес на онлайн покупка.

На базата на направения анализ бяха определени целта и задачите на дипломната работа, които ще бъдат използвани като основа при проектирането и реализацията на системата в следващите глави.

Глава 2. Проект на решението

2.1. Архитектурен модел

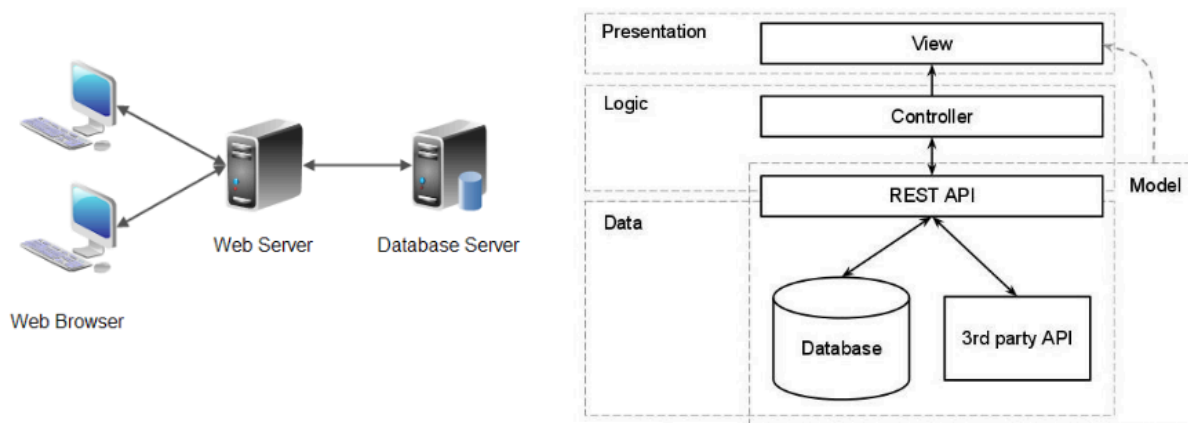
За разработката на TrailBlaze е използван архитектурният модел Model-View-Controller (MVC). Този модел разделя приложението на три основни части, които работят заедно, но изпълняват различни функции. Това разделение улеснява разработката, защото промените в една част не засягат останалите.

Трите архитектурни компонента са:

- **Моделът (Model)** – отговаря за данните в системата. Той комуникира с базата данни и съдържа логиката за това как данните се съхраняват, променят и извличат. Всички основни обекти - потребители, продукти, поръчки, количка, wishlist и адреси за доставка - се управляват от модела. Чрез него се осъществява връзката между сървъра и базата данни.
- **Изгледът (View)** – отговаря за това, което потребителят вижда на екрана. Той представлява потребителският интерфейс на системата. Когато моделът извлече данни от базата, те се предават на изгледа, който ги форматира и показва по подходящ начин. В TrailBlaze изгледът включва всички страници - начална страница, продуктов каталог, детайли за продукт, количка, wishlist, страница за маршрути, AI препоръки и др.
- **Контролерът (Controller)** – служи като посредник между модела и изгледа. Той приема заявките от потребителя (например кликане върху бутон или изпращане на форма), обработва ги, извиква съответните функции на модела и връща резултата към изгледа. Контролерът не съдържа бизнес логика - той само пренасочва заявките и отговорите.

Когато потребител посети сайта, браузърът изпраща заявка към сървъра. Контролерът приема заявката, извлича нужните данни чрез модела и ги предава на изгледа. Изгледът генерира HTML страницата, която потребителят вижда. При извършване на действие (например добавяне на продукт в количката), процесът се повтаря - контролерът обработва действието, моделът променя данните, а изгледът показва актуализираната информация.

Този подход гарантира, че системата е организирана и лесна за поддръжка. Всеки компонент има ясна функция и може да се променя независимо от останалите.



Фигура 1 - Принцип на работата на MVC модела

2.2. Функционален модел

Функционалният модел описва всички действия, които потребителите могат да извършват в системата TrailBlaze. Те са разделени според това дали потребителят е регистриран или не.

Действия за всички потребители (гости)

Всеки посетител на сайта може да:

- Разглежда продукти - преглед на всички налични продукти, разделени в категории (мъже, жени, обувки, промоции)
- Филтрира продукти - търсене по тип, размер, цена и пол за по-лесно намиране на подходяща екипировка
- Разглежда детайли за продукт - виждане на снимки, налични размери, цветове, таблица с размери
- Търси маршрути - търсене на туристически маршрути по град, парк или регион
- Разглежда карта - визуализиране на маршрута върху интерактивна карта с информация за трудност и разстояние
- Проверява прогноза за време - за избран маршрут и дата се показва метеорологична прогноза

- Използва AI Gear Advisor - получава препоръка за подходяща екипировка въз основа на маршрута и времето
- Добавя продукти в количка - избира продукти за покупка
- Добавя продукти в wishlist - запазва продукти за по-късно разглеждане
- Регистрира се - създава потребителски профил
- Допълнителни действия за регистрирани потребители

След създаване на профил, потребителите могат още да:

- Влизат в системата - чрез имейл и парола
- Редактират профила си - промяна на име, имейл или парола
- Финализират поръчка - попълват адрес за доставка и завършват покупката
- Плащат онлайн - чрез интегрираната платежна система Stripe
- Проследяват поръчки - следят статуса на поръчките си в реално време
- Преглеждат история - виждат всички направени досега поръчки
- Преместват продукти - между количка и wishlist

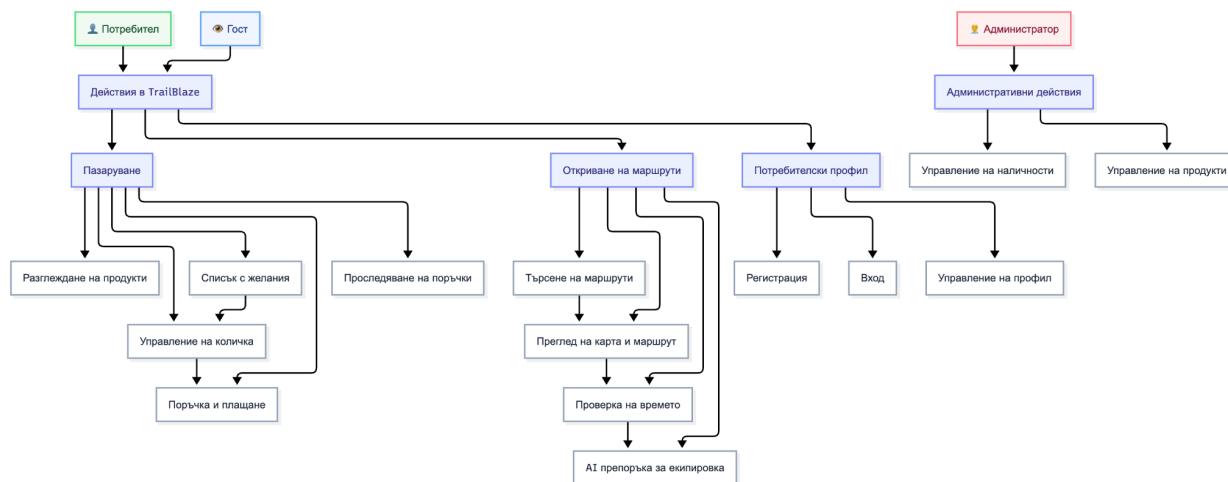
Действия за администратор

Администраторът има допълнителни права за управление на продуктите:

- Управлява наличност - преглед на всички продукти и техните количества по размер
- Коригира количества - увеличава или намалява наличните бройки
- Премахва размери - изтрива конкретен размер от продукт

TrailBlaze съчетава стандартните функции на онлайн магазин с уникални възможности за планиране на туристически преходи. Системата позволява на потребителя да премине през целия процес - от избор на маршрут и проверка на времето, през получаване на AI препоръка за екипировка, до закупуване на продуктите и проследяване на поръчката.

На фигура 2 е представена use-case диаграма, която визуализира всички описани функционалности.



Фигура 2 - Обща use-case диаграма

2.2.1. Сценарии

Сценарий 1: Регистриране на нов профил

Първоначален поток:

1. Потребителят отваря началната страница и натиска бутона "Регистрация".
2. Системата визуализира форма с полета за име, имейл и парола.
3. Потребителят попълва данните и натиска "Създай профил".
4. Системата проверява дали имейлът е свободен. Ако е, създава профил и изпраща имейл за потвърждение. Ако имейлът съществува, преминава към алтернативен поток A1.
5. Потребителят потвърждава имейла си чрез получения линк.
6. Системата активира профила и автоматично вписва потребителя.
7. Сценарият приключва.

Алтернативен поток A1:

1. Системата извежда съобщение "Този имейл адрес вече е регистриран".
2. Потребителят може да влезе в съществуващия си профил.
3. Сценарият приключва.

Сценарий 2: Търсене на туристически маршрут

Първоначален поток:

1. Потребителят отваря страницата "Маршрути".
2. Системата визуализира поле за търсене и бутони за бърз избор.
3. Потребителят въвежда име на град, парк или регион.
4. Системата извлича маршрути от OpenStreetMap и визуализира резултатите.
5. Потребителят избира конкретен маршрут.
6. Системата зарежда детайлна страница с карта и информация за маршрута.
7. Сценарият приключва.

Алтернативен поток A1:

1. Системата не намира маршрути и извежда съобщение "Няма намерени маршрути".
2. Потребителят променя търсенето.
3. Сценарият приключва.

Сценарий 3: Използване на AI Gear Advisor

Първоначален поток:

1. Потребителят отваря детайлна страница на избран маршрут.
2. Системата показва информация за маршрута и календар за избор на дата.
3. Потребителят избира дата за прехода.
4. Системата автоматично извлича прогноза за времето.
5. Потребителят натиска "Вземи AI препоръка".

6. Системата изпраща данни към Claude и получава препоръка за екипировка.
7. Системата визуализира препоръчаните продукти.
8. Сценарият приключва.

Алтернативен поток A1(невалидна дата):

1. Потребителят избира дата в миналото.
2. Системата извежда съобщение "Моля, изберете валидна дата в бъдещето".
3. Потребителят коригира избора.
4. Сценарият продължава от стъпка 4.

Сценарий 4: Добавяне на продукт в количката

Първоначален поток:

1. Потребителят разглежда продукт и влиза в детайлната му страница.
2. Потребителят избира размер и количество.
3. Потребителят натиска "Добави в количката".
4. Системата проверява наличността. Ако продуктът е наличен, го добавя и показва известие. Ако не е, преминава към алтернативен поток A1.
5. Броят в иконата на количката се обновява.
6. Сценарият приключва.

Алтернативен поток A1:

1. Системата извежда съобщение "Избраният размер не е наличен".
2. Потребителят избира друг размер или по-малко количество.
3. Сценарият се връща към стъпка 2.

Сценарий 5: Финализиране на поръчката

Първоначален поток:

1. Потребителят отваря количката и натиска "Продължи към плащане".

2. Системата проверява дали потребителят е влязъл в профила си. Ако не е, го пренасочва към входа.
3. Потребителят попълва адрес за доставка.
4. Потребителят натиска "Продължи към плащане".
5. Системата пренасочва към Stripe за въвеждане на данни за карта.
6. При успешно плащане, системата създава поръчката и изпраща имейл потвърждение.
7. Потребителят може да проследява поръчката си.
8. Сценарият приключва.

Алтернативен поток A1(неуспешно плащане):

1. Системата на Stripe връща грешка.
2. Системата извежда съобщение за причината.
3. Потребителят опитва отново с други данни.
4. Сценарият се връща към стъпка 5.

Сценарий 6: Административно управление на наличност

Първоначален поток:

1. Администраторът влиза в системата.
2. Системата разпознава администратор и показва допълнително меню.
3. Администраторът избира "Администраторски панел".
4. Системата визуализира списък с продукти и наличности по размер.
5. Администраторът избира продукт за редакция.
6. Администраторът коригира количествата и натиска "Запази".
7. Системата проверява валидността на данните. При коректни данни, запазва промените. При некоректни, преминава към алтернативен поток A1.
8. Системата показва съобщение за успешно обновена наличност.
9. Сценарият приключва.

Алтернативен поток A1:

1. Системата извежда съобщение "Моля, въведете валидни числа".
2. Администраторът коригира стойностите.
3. Сценарият се връща към стъпка 6.

Сценарий 7: Проследяване на поръчка

Първоначален поток:

1. Потребителят влиза в профила си.
2. Потребителят избира "Моите поръчки".
3. Системата визуализира списък с всички поръчки.
4. Потребителят избира поръчка за преглед.
5. Системата показва текущия статус (Processing, Packing, In Transit, Delivering, Delivered).
6. Системата автоматично обновява статуса на всеки 30 секунди.
7. Сценарият приключва.

2.3. Проектиране на интерфейса

В етапа на проучване на съществуващите решения установихме, че голяма част от онлайн магазините за туристическа екипировка разполагат с остарели и неинтуитивни интерфейси. Потребителите често се затрудняват да намерят нужната информация, а навигацията между отделните раздели не е достатъчно ясна. В TrailBlaze подходът е различен – интерфейсът е проектиран така, че да бъде максимално изчистен, съвременен и лесен за използване.

За изграждането на потребителския интерфейс са използвани React за създаване на компонентите, Tailwind CSS за стилизиране и DaisyUI за готови, добре проектирани UI елементи. Тази комбинация позволява бързо разработване на адаптивен и визуално привлекателен интерфейс, без да се налага писане на

сложен CSS код. Всички страници са напълно адаптивни и работят коректно както на десктоп, така и на мобилни устройства.

При изграждането на страниците са следвани основните принципи за добър потребителски интерфейс:

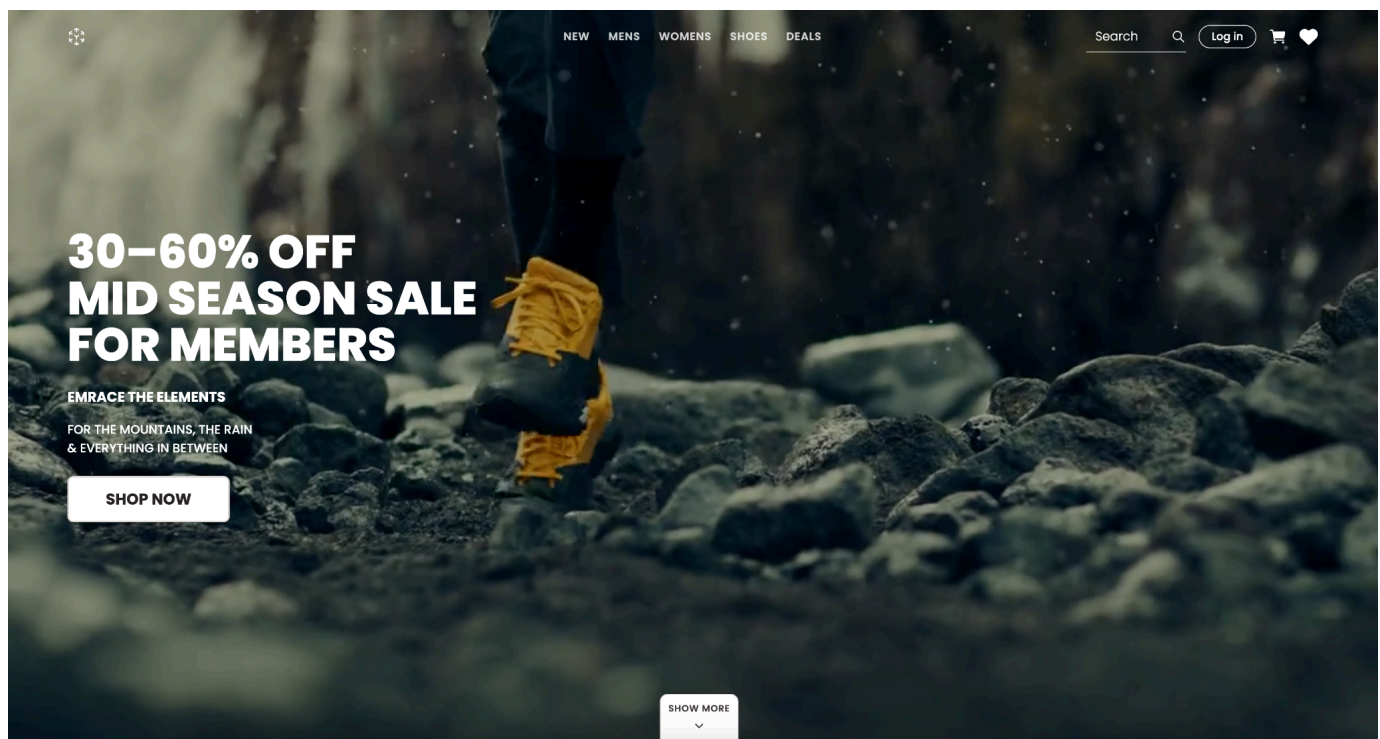
- **Изчистен дизайн** – избягва се използването на излишни елементи. Информацията е представена по начин, който не натоварва зрението на потребителя.
- **Последователност** – всички страници следват единен стил. Бутоните, картите, полетата и другите елементи изглеждат по един и същ начин в цялата система, което улеснява навигацията.
- **Целенасоченост в оформлението** – елементите са подредени според тяхната важност. Най-важната информация (като цена, наличност, бутони за действие) е визуално откроявана, а допълнителните детайли са разположени на подходящи места.
- **Използване на цветове и контрасти** – цветовете са подбрани така, че да създават усещане за природа и планина. Светли фонове тонове се съчетават с акцентни цветове за бутони и важни елементи.
- **Добра типография** – шрифтовете са с подходящ размер и стил, което осигурява добра четливост и визуална йерархия.
- **Информирание на потребителя** – при всяко действие (добавяне в количка, успешна поръчка, грешка и др.) системата показва ясни известия, така че потребителят винаги знае какво се случва.

Структура на интерфейса:

- Хедър – фиксиран в горната част, съдържа логото на TrailBlaze и навигационно меню с връзки към основните раздели: Продукти, Маршрути, AI Advisor, както и икони за количка, wishlist и профил.
- Основна част – съдържанието на всяка страница е организирано в ясни секции, като оформлението варира според типа на страницата – каталог, детайли за продукт, търсене на маршрути и др.

- Футър – разположен в долната част, съдържа информация за контакт и връзки към социалните мрежи.

Вземайки всичко това под внимание, интерфейсът на TrailBlaze осигурява приятно и ефективно потребителско изживяване. На фигура 3 е показан скриншот на началната страница на системат



2.4. Организация на данните

2.4.1. Entity Relationship Diagram

Проектирането на база от данни започва от анализирането на предметната област и след това изготвянето на конкретни модели и връзката им с конкретни типове данни. Моделът Обект-връзка е изключително известен и се използва за графичното представяне на база от данни чрез съответните ER диаграми. Основните компоненти в една такава диаграма са:

- Обект (entity) – наименованието на реален обект от предметната област, например онлайн магазин за туристическа екипировка.
- Клас обекти – това е множеството от обекти със сходни и общи свойства, например продукти, потребители, поръчки.
- Атрибути (attributes) – представляват свойствата на всеки обект в даден клас. Например всеки продукт има име, цена, категория, наличност и снимка. Всеки обект се идентифицира с „ключ“, който представлява уникална стойност.
- Характеризиращ обект – обекти от предметната област, които не могат да съществуват самостоятелно. Например, артикулите в количката не могат да съществуват без самата количка.
- Подклас обекти – обекти, които притежават всички свойства на даден базов обект, но имат и свои допълнителни. Връзката се изобразява чрез триъгълник.

2.4.2. База данни

„Релационната база данни“ е тип база данни, която съхранява данните в таблици, подредени в редове, наричани записи, и колони. Тя позволява създаването на релации между отделните таблици, които осигуряват възможност ефективно да се съхранява огромно количество информация и ефективно да се извлича специфична такава. Софтуерът, който се използва за визуализирането, организирането и управлението на този тип база данни се нарича система за управление на релационни база данни (СУРБД). В TrailBlaze за управление на базата данни е използвана PostgreSQL.

Основните характеристики на една релационна база данни са:

- Релация (relation) – това е терминът, който се използва в терминологията на релационния модел за таблиците, в които се представят данните.
- Домейнът – всички допустими стойности, които могат да бъдат присвоени на един атрибут и техните характеристики. Например, полето за имейл може да приема само валидни имейл адреси.
- Първичен ключ – поле от таблицата, което определя еднозначно записите от таблицата.
- Релации (връзки) – съпоставянето на запис от една таблица със записи от друга таблица. Съществуват няколко вида релации:

Видове връзки между обектите:

- „Едно към едно“ (1:1) – един запис от таблица съответства на точно един запис от друга таблица. Например, един потребител има една количка.
- „Едно към много“ (1:M) – на един запис от таблица съответстват много други от друга таблица. Например, един потребител може да има много поръчки.
- „Много към много“ (M:M) – на един запис от таблица съответстват много записи от друга и обратно. Например, един продукт може да бъде част от много поръчки, а една поръчка може да съдържа много продукти. Този тип връзка се реализира чрез междинна таблица (OrderItem).

Базата данни на TrailBlaze се състои от 9 таблици, като повечето от тях съдържат външни ключове и референции към записи от други таблици. На фигура 5 е представена структурата на базата данни с всички таблици.

Списък на таблиците в базата данни:

User – съхранява информация за потребителите: име, имейл, парола (хеширана), дата на регистрация, статус на верификация.

COLUMNS	Add column
email	TEXT NOT NULL
password	TEXT NOT NULL
createdAt	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
id	SERIAL PRIMARY KEY
additionalEmails	TEXT[]
country	TEXT
gender	TEXT
language	TEXT

Product – съхранява информация за продуктите: име, цена, описание, категория, снимки, наличност по размер, пол (мъже/жени).

COLUMNS	Add column
id	SERIAL PRIMARY KEY
title	TEXT NOT NULL
price	DOUBLE PRECISION NOT NULL
☐ availableSizes	TEXT[]
category	TEXT NOT NULL
gender	TEXT NOT NULL
productImages	TEXT[]
description	TEXT NOT NULL
inStock	BOOLEAN NOT NULL
details	JSONB NOT NULL
fit	TEXT NOT NULL
sizeGuide	JSONB NOT NULL
color	TEXT NOT NULL
discount	INTEGER
stock	JSONB NOT NULL

Cart – обвързва потребителя с неговата количка. Свързана с User чрез външен ключ.

COLUMNS [Add column](#)

☐	id	SERIAL PRIMARY KEY
	userId	INTEGER NOT NULL
☐	createdAt	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
	updatedAt	TIMESTAMP NOT NULL

CartItem – съдържа продуктите в количката: продукт, размер, количество. Свързана с Cart и Product.

COLUMNS [Add column](#)

☐	id	SERIAL PRIMARY KEY
	cartId	INTEGER NOT NULL
	productId	INTEGER NOT NULL
☐	size	TEXT NOT NULL
	color	TEXT NOT NULL
	availableQuantity	INTEGER NOT NULL
	orderQuantity	INTEGER NOT NULL

Wishlist – обвързва потребителя с неговия списък с желани продукти. Свързана с User.

COLUMNS [Add column](#)

	id	SERIAL PRIMARY KEY
	userId	INTEGER NOT NULL
	createAt	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
	updateAt	TIMESTAMP NOT NULL

WishlistItem – съдържа продуктите в wishlist-a: продукт, размер. Свързана с Wishlist и Product.

COLUMNS [Add column](#)

	id	SERIAL PRIMARY KEY
	wishlistId	INTEGER NOT NULL
	productId	INTEGER NOT NULL
	size	TEXT NOT NULL
	color	TEXT NOT NULL
	availableQuantity	INTEGER NOT NULL

Order – съхранява информация за поръчките: потребител, адрес за доставка, обща сума, статус, дата на създаване.

COLUMNS Add column	
id	SERIAL PRIMARY KEY
userId	INTEGER NOT NULL
total	DOUBLE PRECISION NOT NULL
status	TEXT NOT NULL DEFAULT 'paid'
paymentId	TEXT NOT NULL
createdAt	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
updatedAt	TIMESTAMP NOT NULL
deliveryAddressId	INTEGER

OrderItem – съдържа продуктите във всяка поръчка: продукт, размер, количество, цена. Свързана с Order и Product.

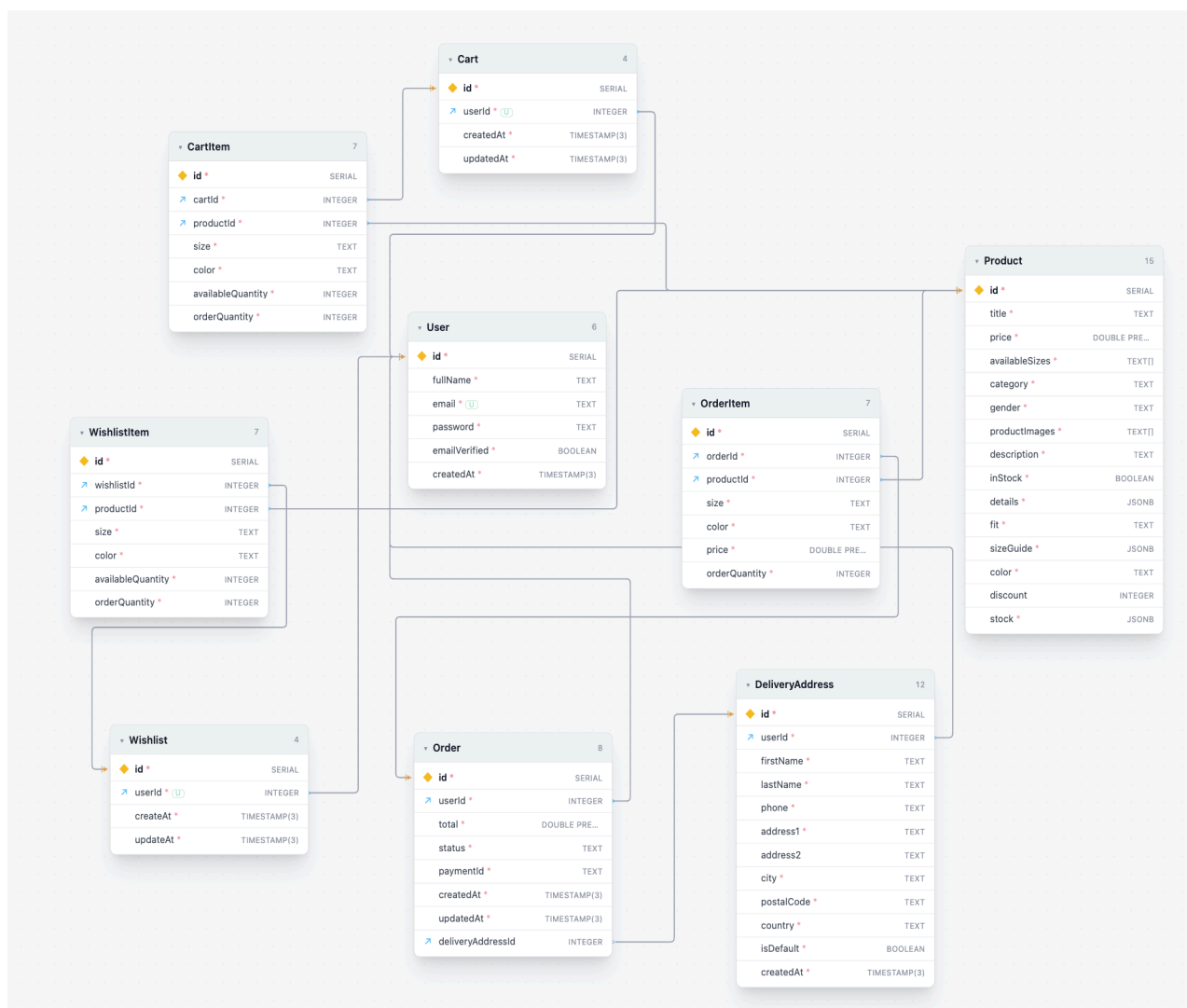
COLUMNS Add column	
id	SERIAL PRIMARY KEY
orderId	INTEGER NOT NULL
productId	INTEGER NOT NULL
size	TEXT NOT NULL
color	TEXT NOT NULL
price	DOUBLE PRECISION NOT NULL
orderQuantity	INTEGER NOT NULL

DeliveryAddress – съхранява адресите за доставка: потребител, държава, град, улица, пощенски код. Свързана с User.

COLUMNS Add column	
id	SERIAL PRIMARY KEY
userId	INTEGER NOT NULL
firstName	TEXT NOT NULL
lastName	TEXT NOT NULL
☐ phone	TEXT NOT NULL
address1	TEXT NOT NULL
address2	TEXT
city	TEXT NOT NULL
postalCode	TEXT NOT NULL
country	TEXT NOT NULL
isDefault	BOOLEAN NOT NULL DEFAULT false
createdAt	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP

Тази структура осигурява ефективно съхранение на всички данни, необходими за работата на системата, и позволява бързо извличане на информация при различни заявки.

На фигура 3 е показана структурата на базата данни с всички таблици и техните полета.



Фигура 3 - Структура на базата данни на TrailBlaze

Глава 3. Избор на средства и описание на реализацията

3.1. Избор на средства

За реализацията на TrailBlaze е използван набор от съвременни уеб технологии,

които осигуряват висока производителност, лесна поддръжка и добро потребителско изживяване. Изборът на средства е направен след проучване на съществуващите решения и съобразяване със специфичните изисквания на Системата.

3.1.1. Избор на език

1.1.1 TypeScript

TypeScript е основният език, използван в TrailBlaze, както за клиентската, така и за сървърната част. Това е надмножество на JavaScript, което добавя статична типова система. Използването на TypeScript осигурява няколко съществени предимства:

- Типова безопасност – грешките се откриват още по време на разработка, а не по време на изпълнение на кода.
- По-добра поддръжка – кодът е по-четлив и лесен за разбиране, което улеснява бъдещи промени и разширения.
- Автоматично довършване – съвременните редактори предоставят интелигентно допълване на кода, което ускорява разработката.
- Лесно рефакториране – промените в структурата на кода могат да се извършват по-безопасно.

Използването на TypeScript е особено полезно при по-големи проекти като TrailBlaze, където има много взаимодействащи си компоненти и модели.

1.1.2 HTML и CSS

HTML5 е използван за структуриране на съдържанието на уеб страниците. Той предоставя семантични елементи, които подобряват както четливостта на кода, така и оптимизацията за търсачки.

CSS3 е използван за визуалното оформление на страниците. В комбинация с Tailwind CSS, който е използван като основна CSS библиотека, се постига бързо и последователно стилизиране на всички компоненти. Tailwind предлага:

- Utility-first подход – вместо да се пише персонализиран CSS за всеки елемент, се използват предварително дефинирани класове.
- Адаптивност – вградена поддръжка за различни устройства.
- Оптимизация – автоматично премахване на неизползваните стилове при компилация.

За по-сложни компоненти е използвана и библиотеката DaisyUI, която предоставя готови UI компоненти, базирани на Tailwind CSS.

1.1.3 JavaScript (React)

React е основната библиотека за изграждане на потребителския интерфейс. Това е една от най-популярните и добре поддържани библиотеки за изграждане на динамични уеб приложения.

Основните предимства на React в контекста на TrailBlaze са:

- Компонентен подход – интерфейстът е разделен на независими, преизползваеми компоненти. Например, ProductCard, TrailCard, CartSummary и др.
- Виртуален DOM – осигурява бързо обновяване на интерфейса без презареждане на страницата.
- Еднопосочен поток на данните – улеснява проследяването на промените и отстраняването на грешки.
- Голяма екосистема – богат набор от библиотеки и инструменти за разширяване на функционалността.

За разработката е използвана React 19, която въвежда подобрения в производителността и нови функционалности като React Compiler.

1.1.4 Node.js и Express.js

За сървърната част е използван Node.js – среда за изпълнение на JavaScript извън браузъра. Node.js е избран поради:

- Висока производителност – базиран е на V8 двигателя на Google.
- Асинхронен модел – подходящ за обработка на множество едновременни заявки.

- Единен език – използва се JavaScript както на клиента, така и на сървъра, което улеснява разработката.

Като уеб фреймуърк за Node.js е използван Express.js. Той предоставя лесен и гъвкав начин за дефиниране на маршрути, обработка на заявки и управление на middleware.

1.1.5 PostgreSQL

За съхранение на данните е използвана релационната база данни PostgreSQL. Тя е една от най-популярните и надеждни СУРБД, използвана в множество уеб приложения.

Основните причини за избора на PostgreSQL са:

- Надеждност – доказана стабилност и поддръжка на ACID транзакции.
- Разширяемост – поддръжка на JSON, географски данни и други типове.
- Производителност – оптимизирана за работа с големи обеми данни.
- Отворен код – безплатна и с активна общност.

Достъпът до базата данни се осъществява чрез Prisma ORM, който осигурява типобезопасни заявки и улеснява миграциите.

1.1.6 Vite

За изграждане на клиентската част е използван Vite – съвременен инструмент за сборка, който заменя по-стари решения като Webpack.

Предимствата на Vite включват:

- Бързо стартиране – почти мигновено зареждане на средата за разработка.
- Бързи компилации – използва ES модули за по-бързо презареждане.
- Оптимизирани сборки – автоматично разделяне на кода и оптимизация за производствена среда.

3.1.2. Избор на софтуерен фреймуърк

За реализацията на специфичните функционалности на TrailBlaze са интегрирани няколко външни услуги. За обработка на плащания е избрана Stripe, която осигурява сигурно приемане на кредитни и дебитни карти. За търсене на туристически маршрути са използвани OpenStreetMap и Overpass API, които предоставят безплатна географска информация за маршрути по целия свят. За изчисляване на маршрутни направления и издигане системата използва OpenRouteService. Метеорологичната прогноза се получава чрез Open-Meteo – безплатен API за време. За генериране на AI препоръки за екипировка е използван Claude на Anthropic, който анализира маршрута и времето, за да предложи подходящи продукти. За изпращане на имейли (верификация, потвърждение на поръчки) служи Resend, а за визуализиране на маршрути със снимки – Pexels API.

3.1.3. Избор на уеб сървър

За локална разработка е използван стандартен инструментариум за Node.js и React проекти. Управлението на пакетите се извършва чрез npm, който осигурява по-бърза и ефективна работа с зависимости. За едновременно стартиране на клиентската и сървърната част е използван Concurrently. Системата за контрол на версиите е Git, като кодът се съхранява в GitHub

3.1.4. Инфраструктура и разполагане

За разполагане на системата в производствена среда са използвани две платформи. Фронтенд частта е разположена на Vercel, който осигурява автоматично разполагане при промяна в кода, вградени SSL сертификати и CDN за бързо зареждане. Бекенд частта работи на Render, който предлага лесно разполагане на Node.js приложения с автоматично мащабиране. Базата данни PostgreSQL е хоствана в Neon – сървърна PostgreSQL услуга с автоматично архивиране и висока наличност.

3.2. Описание на реализацията

Тази част описва как проектните решения от глава 2 са реализирани на ниво програмен код. Разгледани са организацията на сървърната част, приложно-програмният интерфейс, контролът на достъпа, работата с базата данни и интеграциите с външни услуги.

3.2.1. Организация на сървърната част

Сървърната част е изградена върху Express и следва слоевата организация, която отделя маршрутизирането от бизнес логиката и от достъпа до данните. Изходният код е разпределен в следните директории:

`routes` - дефинира адресите и свързва всеки от тях с функция от съответния контролер. Съдържа седемнадесет модула, по един за всяка функционална област.

`controllers` - съдържа обработващите функции. Всяка приема заявка, извършва необходимите проверки, обръща се към базата данни или към външна услуга и връща отговор.

`middlewares` - междинни слоеве, изпълнявани преди контролера. Тук са проверката на сесията и проверката на административни права.

`services` - помощни модули с логика, която не принадлежи на конкретен контролер, например обновяването на статуса на поръчка.

`config` - инициализация на клиента към базата данни, генериране на токени и настройка на услугата за изпращане на писма.

Основният файл `index.js` регистрира междинните слоеве за обработка на тялото на заявката, за политиката за споделяне на ресурси между източници и за четене на бисквитки, след което монтира отделните групи маршрути под общ префикс `/api`.

```

app.use("/api/user", authRouter);
app.use("/api/cart", cartRouter);
app.use("/api/products", productsRouter);
app.use("/api/wishlist", wishlistRouter);
app.use("/api/checkout", checkoutRouter);
app.use("/api/orders", ordersRouter);
app.use("/api/ors", orsRouter);
app.use("/api/overpass", overpassRouter);
app.use("/api/open-meteo", openMeteoRouter);
app.use("/api/ai", aiRouter);
app.use("/api/admin", adminRouter);

```

Фигура 3.1 - Монтиране на групите маршрути

3.2.2. Приложно-програмен интерфейс

Обменът между клиентската и сървърната част се извършва изцяло чрез JSON. Адресите следват принципа ресурсът да се идентифицира от пътя, а действието от метода на заявката. Таблица 3.1 представя основните групи ресурси.

Група	Метод и път	Предназначение	Достъп
Профил	POST /api/user/register	Създаване на профил	Публичен
Профил	POST /api/user/login	Вписване	Публичен
Профил	GET /api/user/verify-email	Потвърждаване на адрес	Публичен
Профил	GET /api/user/profile	Данни за профила	Сесия
Профил	PUT /api/user/change	Промяна на данни	Сесия
Продукти	GET /api/products	Списък с продукти	Публичен
Количка	GET /api/cart	Съдържание на количката	Сесия
Количка	POST /api/cart/add/:productId	Добавяне на продукт	Сесия
Количка	POST /api/cart/update/:id	Промяна на количество	Сесия
Желани	POST /api/wishlist/add/:productId	Добавяне към желани	Сесия
Поръчки	POST /api/orders/confirm	Потвърждаване на поръчка	Сесия
Плащане	POST /api/checkout/add	Създаване на плащане	Сесия
Маршрути	POST /api/overpass/interpreter	Търсене на пътеки	Публичен
Маршрути	POST /api/ors/hiking-route	Изчисляване на маршрут	Публичен
Време	GET /api/open-meteo/forecast	Прогноза за времето	Публичен
AI	POST /api/ai/suggest	Препоръка за екипировка	Публичен

Наличности	PUT /api/admin/add/:productId	Увеличаване на наличност	Администратор
Наличности	PUT /api/admin/decrease/:productId	Намаляване на наличност	Администратор
Наличности	DELETE /api/admin/remove/:productId/:size	Премахване на размер	Администратор

Таблица 3.1 - Основни ресурси на приложно-програмния интерфейс

Използваните кодове на отговор следват стандартната семантика на протокола HTTP. Код 200 означава успешно изпълнена заявка, 201 - успешно създаден ресурс, 400 - невалиден вход, 401 - липсваща сесия, 403 - недостатъчни права, 404 - несъществуващ ресурс, а 500 - грешка от страна на сървъра.

3.2.3. Контрол на достъпа

Сесията се пренася чрез бисквитка с име `jwt`, поставяна при успешно вписване. Бисквитката е с флаг `httpOnly`, което не позволява четенето ѝ от клиентски скриптове, и с флаг `secure`, който я ограничава до защитена връзка. Срокът ѝ на валидност е седем дни.

Проверката на сесията се извършва от междинния слой `verifyJWT`. Той извлича токена от бисквитката, проверява подписа му и записва декодираните данни в обекта на заявката, откъдето контролерите получават идентификатора на текущия потребител.

Административните операции преминават през втори междинен слой, `requireAdmin`, изпълняван след проверката на сесията. Ролята се чете от базата данни, а не от съдържанието на токена. Това решение е съзнателно: ако ролята се съхранява в токена, отнемането на права би влязло в сила едва след изтичането му, докато при четене от базата данни промяната действа незабавно.

В клиентската част достъпът е ограничен допълнително чрез два компонента за обвиване на маршрути. `ProtectedRoute` пренасочва невластен посетител към страницата за вход, а `GuestRoute` върши обратното, като не позволява на вече властен потребител да отвори формите за вход и регистрация. Тази защита е единствено за удобство на потребителя. Действителното ограничение се налага на сървъра, тъй като клиентският код е достъпен за промяна от посетителя.

3.2.4. Работа с базата данни

Достъпът до данните се осъществява чрез `Prisma`, който предоставя типизиран интерфейс към `PostgreSQL` и премахва нуждата от ръчно съставяне на заявки. Схемата се описва декларативно, а промените се нанасят чрез миграции, което

запазва историята на изменения в структурата и позволява тя да бъде възпроизведена в друга среда.

Заявките се съставят чрез методите на генерирания клиент, а стойностите се подават като параметри, а не чрез слепване на низове. Така рискът от вмъкване на SQL е изключен на ниво библиотека.

```
const product = await prisma.product.findFirst({
  where: { id: Number(productId) },
});
```

Фигура 3.2 - Пример за параметризирана заявка чрез Prisma

Наличностите по размери се съхраняват в поле от тип JSONB в таблицата Product. Решението позволява различните продукти да имат различни набори от размери, включително числови за панталони и буквени за горно облекло, без да се налага отделна таблица за всяка схема на размери.

3.2.5. Интеграция с картографски услуги

Функционалността за откриване на маршрути използва три отделни услуги, всяка с различна роля.

Overpass API служи за търсене на туристически пътеки в зададена правоъгълна област. Заявките се съставят на езика Overpass QL и се изпращат през сървъра, а не директно от браузъра, за да не се разкрива структурата на заявките и за да може отговорът да бъде проверен преди предаване към клиента.

OpenRouteService изчислява действителния маршрут между избрани точки по профил foot-hiking, съобразен с пешеходни пътеки. Услугата връща данни във формат GeoJSON, съдържащи геометрията на маршрута и обобщение с дължина и предполагаема продължителност.

Leaflet, чрез обвивката react-leaflet, визуализира получената геометрия върху карта с подложка от OpenStreetMap. Изборът на Leaflet вместо търговска картографска услуга е продиктуван от липсата на такса за използване и от това, че данните на OpenStreetMap покриват туристическите пътеки в България значително по-добре от масовите алтернативи.

Ключовете за достъп до OpenRouteService се съхраняват в променливи на средата и не напускат сървъра. Клиентската част никога не се обръща пряко към външната услуга.

3.2.6. Прогноза за времето

Метеорологичните данни се получават от Open-Meteo, услуга, която не изисква ключ за достъп при некомерсиална употреба. За избрана точка и дата се извличат температура, количество валежи и скорост на вятъра. Отделен ресурс връща надморската височина, която има значение при планински маршрути, където условията се различават съществено от тези в подножието.

Получените стойности изпълняват двойна роля: показват се на потребителя и служат за вход на модула за препоръка на екипировка.

3.2.7. Препоръка за екипировка

Модулът за препоръка използва езиков модел, достъпван през интерфейса на Anthropic. При заявка се извличат всички продукти от базата данни и се съставя списък с наименование, категория и адрес на страницата на всеки от тях. Този списък се подава на модела заедно с името на маршрута, неговата трудност и текущата метеорологична прогноза.

Съществено ограничение в системната инструкция е изискването моделът да препоръчва изключително продукти от подадения списък. Без такова ограничение езиковият модел би предложил произволни изделия, които магазинът не предлага, и препоръката би останала безполезна. Отговорът се изисква в предварително зададена структура, което позволява той да бъде обработен програмно и всяка препоръка да бъде свързана с реална продуктова страница.

3.2.8. Плащане

Плащанията се обработват чрез Stripe. Реализацията използва механизма PaymentIntent: сървърът създава намерение за плащане с изчислената сума в евро и връща на клиента еднократен идентификатор, чрез който браузърът завършва плащането директно към Stripe.

Съществено предимство на този подход е, че данните на платежната карта никога не преминават през сървъра на приложението. Полетата за въвеждане се предоставят от библиотеката на Stripe и се изобразяват в изолирана рамка, принадлежаща на домейна на доставчика. Така приложението няма достъп до номера на картата и изискванията на стандарта PCI DSS към него остават значително по-леки.

Съдържанието на поръчката се предава в полето metadata на намерението за плащане в съкратен вид, съдържащ идентификатор на продукт, количество,

размер, цвят и цена. Това позволява поръчката да бъде възстановена при потвърждаване, без да се разчита на състояние, съхранено в браузъра.

3.2.9. Проследяване на поръчки

След успешно плащане поръчката се записва със статус, който впоследствие се обновява. Смяната на статусите е реализирана чрез отложени изпълнения в рамките на процеса на сървъра, които последователно превеждат поръчката през състоянията на опаковане, транспортиране и доставяне.

Решението симулира работата на действителна логистична система и е подходящо за целите на демонстрацията, но има известно ограничение: отложените изпълнения съществуват единствено в паметта на процеса. При рестартиране на сървъра насрочените обновявания се губят и поръчката остава в текущото си състояние. В производствена среда тази задача би следвало да се изнесе към опашка от задачи или към планировчик, съхраняващ състоянието си извън процеса. Възможността е разгледана в глава 5.

3.2.10. Организация на клиентската част

Клиентската част е изградена с React и се компилира чрез Vite. Изходният код е разделен на две основни директории. В `pages` се намират компонентите, съответстващи на отделните страници: начална страница, категория, страница на продукт, количка, маршрути, профил, поръчки и административен панел. В `shared` са изнесени многократно използваните елементи: заглавна част, долен колонтитул, стъпките на поръчката, индикаторите за зареждане и обработката на грешки.

Маршрутизирането се осъществява чрез `react-router-dom`. Част от маршрутите са обвити в компоненти, които налагат ограничения за достъп, както е описано в точка 3.2.3.

```
<Route path="/trails" element={<Trails />} />
<Route path="/trails/:id" element={<TrailDetails />} />
<Route element={<ProtectedRoute />>
  <Route path="/orders" element={<Orders />} />
  <Route path="/profile" element={<Profile />} />
  <Route path="/admin" element={<AdminPage />} />
</Route>
```

Фигура 3.3 - Дефиниране на маршрутите в клиентската част

Оформлението използва Tailwind CSS заедно с надстройката DaisyUI. Подходът, при който стиловете се задават чрез класове с помощно предназначение, позволява оформлението да се променя, без да се поддържат отделни файлове със стилове за всеки компонент, и подпомага последователността, описана в точка 2.3.

Обръщенията към сървъра са централизирани в отделен модул, който добавя основния адрес и указва бисквитките да се изпращат заедно със заявките. Съобщенията към потребителя при успешно или неуспешно действие се извеждат чрез react-hot-toast, което изпълнява изискването за обратна връзка, формулирано в точка 2.3.

Глава 4. Описание на тестването на предложеното решение

4.1. Подход към тестването

Тестването на TrailBlaze е проведено ръчно, без автоматизиран набор от тестове. Изборът е продиктуван от характера на системата: значителна част от функционалността е съсредоточена в потребителския интерфейс и в интеграциите с външни услуги (OpenRouteService, Overpass, Open-Meteo, Stripe, Resend), чието поведение зависи от състоянието на трети страни и не може да бъде възпроизведено надеждно с модулни тестове без изграждане на симулационен слой.

Тестовите случаи са изведени пряко от use-case диаграмата, представена на фигура 2 в глава 2. Всеки листов случай на употреба е превърнат в поне един сценарий, като към тях са добавени и отрицателни случаи, проверяващи поведението при невалиден вход и при липса на права.

Използвани са и трите класически подхода към тестването:

Затворена кутия (black box) - системата се разглежда отвън, през интерфейса, без предположения за вътрешната реализация. Проверява се съответствието между наблюдаваното поведение и описаните в глава 2 сценарии.

Частично отворена кутия (grey box) - действие се извършва през интерфейса или през приложно-програмния интерфейс, след което резултатът се проверява в базата данни или в отговора на сървъра. Този подход открива несъответствия, които на екрана изглеждат коректно.

Отворена кутия (white box) - анализира се конкретен програмен код и се построяват тестове, покриващи отделните му клонове.

Тестовата среда е локална: клиентската част работи чрез Vite на порт 5173, сървърната част чрез Express на порт 4996, а базата данни е PostgreSQL, достъпвана чрез Prisma. В базата са налични 60 продукта. Всички променени по време на тестването данни са върнати в изходното им състояние.

4.2. Тестване по метода на затворената кутия

Сценариите в таблица 4.1 са изведени от четиринадесетте листови случая на употреба в use-case диаграмата и обхващат трите актьора: гост, регистриран потребител и администратор.

№	Сценарий	Актьор	Очакван резултат
1	Разглеждане на продукти по категория	Гост	Списък с продукти, филтрите работят
2	Добавяне в количка без избран размер	Гост	Съобщение за грешка
3	Добавяне в количка с размер и цвят	Потребител	Продуктът се появява в количката
4	Количество над наличното	Потребител	Ограничаване до наличната бройка
5	Списък с желания без вход	Гост	Пренасочване към страницата за вход
6	Регистрация със зает имейл	Гост	Съобщение за дублиран адрес
7	Вход с грешна парола	Гост	Отказан достъп
8	Поръчка и плащане чрез Stripe	Потребител	Потвърждение и известие по имейл
9	Проследяване на поръчка	Потребител	Актуален статус на поръчката
10	Търсене на маршрути	Гост	Резултати от Overpass
11	Преглед на карта и маршрут	Гост	Карта с начертан маршрут
12	Проверка на прогнозата за времето	Гост	Данни от Open-Meteo
13	AI препоръка за екипировка	Потребител	Списък с препоръчана екипировка
14	Достъп до администраторския панел	Потребител	Отказан достъп

Таблица 4.1 - Сценарии за тестване по метода на затворената кутия

4.3. Тестване по метода на частично отворената кутия

Проверките в тази група са насочени към приложно-програмния интерфейс. Заявките са изпращани директно към сървъра, а резултатът е сверяван със състоянието на базата данни. Таблица 4.2 обобщава проверките на публичните ресурси и на интеграциите с външни услуги.

№	Ресурс	Вход	Код	Наблюдаван резултат
A1	GET /api/products	-	200	60 продукта, 15 полета
A2	GET /api/open-meteo/forecast	Рила	200	Прогноза върната
A3	GET /api/open-meteo/elevation	Рила	200	Надморска височина върната
A4	POST /api/ors/hiking-route	Алеко - Черни връх	200	4569 м, 84 мин, 141 точки
A5	POST /api/ors/hiking-route	Златни мостове - Алеко	200	6601 м, 142 мин, 210 точки
A6	POST /api/ors/hiking-route	Точка извън пътека	500	Общо съобщение за грешка
A7	POST /api/overpass/interpreter	Валидна Overpass QL заявка	200	15 обекта, 2 именувани пътеки
A8	POST /api/overpass/interpreter	Липсващо поле data	429	Докладвано като rate limit
A9	POST /api/user/login	Грешна парола	400	Отказан достъп
A10	GET /api/user/get_user	Без сесия	401	Отказан достъп
A11	GET /api/user/profile	Без сесия	401	Отказан достъп

Таблица 4.2 - Проверки на приложно-програмния интерфейс

Отделна група проверки е насочена към контрола на достъпа. Целта е да се установи дали ограничението за непотвърден имейл и разделянето на правата между потребител и администратор се спазват на ниво сървър, а не само в потребителския интерфейс. Резултатите са в таблица 4.3.

№	Проверка	Код	Наблюдаван резултат
B1	Регистрация на нов профил	201	Профилът е създаден, върнат е JWT
B2	Вход с непотвърден имейл	403	Достъпът е отказан коректно
B3	GET /api/user/profile с токена от регистрацията	201	Достъпът е разрешен
B4	GET /api/cart с токена от регистрацията	200	Достъпът е разрешен
B5	PUT /api/admin/decrease/3 с обикновен профил	200	Наличността е променена

Таблица 4.3 - Проверки на контрола на достъпа

Проверките B3 и B4 показват, че ограничението, наложено при вход, се заобикаля. Отговорът на заявката за регистрация съдържа валиден токен, издаден преди потвърждаването на адреса, и този токен се приема от междинния слой за проверка. Проверката B5 показва, че същият токен, принадлежащ на новосъздаден и непотвърден профил, дава достъп до административните операции.

4.4. Тестване по метода на отворената кутия

За анализ по метода на отворената кутия е избрана функцията `decreaseSizeAmount` от модула `manageSizeController.js`, която намалява наличността на определен размер с една бройка. Функцията съдържа условен клон, изпълняван само когато наличността достигне нула, и този клон не се покрива при обичайна употреба.

```
const newStockSize = currentStock[size] - 1;
const updatedStock = { ...currentStock };

if (newStockSize <= 0) {
  delete currentStock[size];
} else {
  updatedStock[size] = newStockSize;
}

await prisma.product.update({
  where: { id: Number(productId) },
  data: { stock: updatedStock },
});
```


Фигура 4.1 - Първоначална реализация на намаляването на наличност

Обектът `updatedStock` е плитко копие на `currentStock`, направено преди изтриването. В клона за последната бройка операцията `delete` се прилага върху `currentStock`, тоест върху оригинала, докато в базата данни се записва `updatedStock`, което все още съдържа старата стойност. Следствието е, че последната бройка от даден размер не може да бъде изчерпана.

За проверка е използван продукт 3 с начално състояние на наличностите {"M": 1, "S": 5, "XL": 4}. Размер M се намира точно на границата, при която се задейства проблемният клон, а размер S служи за контролен случай.

№	Действие	Очаквано	Наблюдавано
C1	Намаляване на M (1 → 0)	Ключът M отпада	M остава 1
C2	Намаляване на M, четири последователни опита	Ключът M отпада	M остава 1
C3	Намаляване на S (5 → 4)	S става 4	S става 4

Таблица 4.4 - Покриване на клоновете на `decreaseSizeAmount`

Контролният случай C3 доказва, че функцията работи коректно в общия случай. Дефектът е локализиран единствено в клона за последната бройка, което го прави труден за забелязване при ръчна проверка през интерфейса.

4.5. Открити дефекти

Проведеното тестване откри девет дефекта. Таблица 4.5 ги подрежда по приоритет, определен според това дали дефектът засяга сигурността, коректността на данните или единствено удобството на работа.

№	Дефект	Модул	Приоритет
H3	Заобикаляне на потвърждаването на имейл	<code>authController.js</code>	Висок
H4	Липсва разделяне на правата на достъп	<code>manageSizeRoutes.js</code>	Висок
H8	Потвърждаване на профил чрез познаване на идентификатор	<code>authController.js</code>	Висок

H5	Последната бройка от размер не може да бъде изчерпана	manageSizeController.js	Среден
H9	Писмото за потвърждение се изпраща до фиксиран адрес	authController.js	Среден
H6	Параметърът quantity се игнорира	manageSizeController.js	Нисък
H1	Обобщаване на всички грешки от ORS до код 500	orsController.js	Нисък
H2	Невалидна заявка се докладва като ограничение на честотата	overpassController.js	Нисък
H7	Неизползван и незавършен междинен слой	authMiddleware.js	Нисък

Таблица 4.5 - Открити дефекти по приоритет

Дефектите H3, H4 и H8 образуват обща верига. Профил може да бъде създаден с произволен адрес, токен се получава веднага в отговора на регистрацията, а административните маршрути не проверяват роля. Комбинацията позволява на произволен посетител да променя наличностите на продуктите. Според класификацията OWASP Top Ten това съответства на категория A01:2021 - Broken Access Control.

Тук се проявява и разминаване между проектното решение и реализацията. Сценарий 1 в глава 2 описва, че системата активира профила след потвърждаване на адреса и едва тогава вписва потребителя. Реализацията издава сесия още при регистрацията, преди каквото и да е потвърждение. Именно това разминаване е основната находка от тестването.

4.6. Отстраняване на дефектите

Всички открити дефекти са отстранени. Таблица 4.6 обобщава направените промени.

Дефект	Модул	Промяна
H3	authController.js	Регистрацията не издава токен и не поставя cookie
H4	schema.prisma, authMiddleware.js	Поле role и междинен слой requireAdmin

H8	authController.js	Еднократен 32-байтов токен с 24-часов срок
H5	manageSizeController.js	Изтриването се прилага върху записваното копие
H9	authController.js	Писмото се изпраща до регистрирания се адрес
H6	manageSizeController.js	Стойността на quantity се използва
H7	authMiddleware.js	Премахнат неизползван код

Таблица 4.6 - Направени промени

Корекцията на H5 се състои в прилагане на операцията delete върху копие, което се записва в базата данни:

```
const newStockSize = currentStock[size] - 1;
const updatedStock = { ...currentStock };

if (newStockSize <= 0) {
  delete updatedStock[size];
} else {
  updatedStock[size] = newStockSize;
}
```

Фигура 4.2 - Коригирана реализация на намаляването на наличност

За отстраняването на H4 към модела User е добавено поле role със стойност по подразбиране "user". Ролята се чете от базата данни, а не от токена, за да може отнемането на права да влезе в сила незабавно и да не зависи от съдържанието на вече издаден токен:

```
export const requireAdmin = async (req, res, next) => {
  const user = await prisma.user.findUnique({
    where: { id: req.user.id },
    select: { role: true, emailVerified: true },
  });

  if (!user || !user.emailVerified || user.role !== "admin") {
    return res.status(403).json({ message: "Administrator access required" });
  }

  next();
};
```

Фигура 4.3 - Проверка на административни права

Повторното тестване след нанасянето на промените е обобщено в таблица 4.7. Проверките възпроизвеждат същите действия, извършени преди корекциите.

№	Проверка	Преди	След
П1	Токен в отговора на регистрацията	Издава се	Не се издава
П2	Cookie при регистрация	Поставя се	Не се поставя
П3	Записване на fullName	Празно	Записва се
П4	Потвърждаване чрез идентификатор	Успешно	Отхвърля се
П5	Потвърждаване чрез валиден токен	-	Успешно
П6	Повторна употреба на токена	-	Отхвърля се
П7	Административен ресурс, обикновен профил	200	403
П8	Административен ресурс, администратор	200	200
П9	Достъп до собствена количка	200	200
П10	Изчерпване на последната бройка	Остава 1	Размерът отпада
П11	Добавяне с quantity равно на 5	Добавя 1	Добавя 5

Таблица 4.7 - Резултати от повторното тестване

4.7. Обобщение

Ръчното тестване, изведено от use-case диаграмата, откри девет дефекта, три от които засягат сигурността на системата. Най-съществената находка е, че ограничението за непотвърден имейл, което изглежда работещо при вход, се заобикаля чрез токена, издаван при регистрация, и че административните операции не са отделени от потребителските.

Особено полезен се оказва подходът на частично отворената кутия. Проверките В3, В4 и В5 не биха открили нищо, ако бяха проведени само през потребителския интерфейс, тъй като клиентската част не предлага начин за достигане на административните операции от обикновен профил. Дефектът е видим единствено при директни заявки към сървъра.

Подходът на отворената кутия показва стойността на контролния случай. Намаляването на наличност изглежда изправно при всяка стойност освен последната, поради което дефектът H5 е останал незабелязан при предходната работа по модула.

След нанесените корекции всички единадесет повторни проверки преминават успешно. Остава едно ограничение на средата: изпращащият адрес принадлежи на пясъчника на Resend и доставя писма единствено до адреса на притежателя на акаунта. До регистриране на собствен домейн реална регистрация на външни потребители не е възможна. Това ограничение не произтича от кода и е отбелязано в глава 5.

Глава 5. Бъдеща визия

Настоящата глава очертава посоките, в които TrailBlaze може да бъде развит. Изброените задачи произтичат пряко от ограниченията, установени при разработката и при тестването, описано в глава 4.

5.1. Технически задачи с непосредствен приоритет

Собствен домейн за изпращане на писма. Понастоящем писмата за потвърждение се изпращат от адрес, принадлежащ на пясъчника на доставчика Resend, който доставя единствено до адреса на притежателя на акаунта. Регистрирането на собствен домейн и настройването на записите SPF и DKIM е задължително условие, за да може системата да обслужва външни потребители.

Автоматизирано тестване. Проверките, описани в глава 4, са проведени ръчно. Изграждането на набор от автоматизирани тестове за сървърната част би позволило дефекти като описания в точка 4.4 да бъдат откривани при всяка промяна, а не при целенасочено търсене. Особено подходящи за автоматизация са проверките на контрола на достъпа, тъй като те не зависят от потребителския интерфейс.

Устойчиво обновяване на статуса на поръчките. Както е отбелязано в точка 3.2.9, обновяването се извършва чрез отложени изпълнения в паметта на процеса и се губи при рестартиране. Пренасянето на тази задача към опашка от задачи със съхранявано състояние би направило поведението предвидимо.

Прецизиране на съобщенията за грешки. При заявка към OpenRouteService за точка, край която няма картографирана пътека, услугата връща отказ, който системата предава като обща сървърна грешка. По-точно би било такъв случай да се разпознава като невалиден вход и потребителят да получи указание да избере друга начална точка. Сходно, отговорите на Overpass при синтактично невалидна заявка понастоящем се тълкуват като ограничение на честотата.

5.2. Разширяване на функционалността

Пълноценен административен панел. Описаният в глава 2 случай на употреба "Управление на продукти" понастоящем е реализиран само в частта, отнасяща се до наличностите по размери. Добавянето, редактирането и премахването на самите продукти все още изисква пряка намеса в базата данни.

Кеширане на обръщенията към външни услуги. Резултатите от търсенето на пътеки за дадена област се променят рядко. Съхраняването им за определен период би намалило натоварването върху публичната инфраструктура на Overpass и би ускорило работата на страницата с маршрути.

Потребителски отзиви и оценки за продуктите, както и записване на изминати маршрути в потребителския профил, биха обвързали по-тясно двете основни части на системата - магазина и планирането на преходи.

Многоезичност. Понастоящем интерфейсът е достъпен на един език. Изнасянето на текстовете в отделни ресурсни файлове би позволило добавянето на нови езици без промени в компонентите.

Заклучение

В рамките на дипломната работа е проектирана и реализирана уеб базирана система TrailBlaze, съчетаваща електронен магазин за туристическа екипировка с инструменти за планиране на пешеходни маршрути.

Направеният в глава 1 преглед на съществуващи решения показва, че функционалността на онлайн магазините е добре установена, но че специализираните магазини за туристическа екипировка не предлагат средства, подпомагащи избора според конкретния преход. Именно това наблюдение определи целта на разработката.

В глава 2 са описани архитектурният и функционалният модел на системата, сценариите на употреба и организацията на данните. В глава 3 са обосновани използваните технологии и е описана реализацията: слоестата организация на сървърната част, приложно-програмният интерфейс, контролът на достъпа, работата с базата данни чрез Prisma и интеграциите с картографски, метеорологични и платежни услуги.

Глава 4 представя проведеното тестване. Изведените от диаграмата на случаите на употреба проверки откриха девет дефекта, три от които засягат сигурността на системата. Най-съществената находка е разминаване между проектното решение и реализацията: описаният в глава 2 сценарий изисква профилът да се активира след потвърждаване на адреса, докато реализацията издаваше сесия още при регистрацията. Всички открити дефекти са отстранени, а нанесените промени са проверени с единадесет повторни теста.

Целта и задачите, поставени в точка 1.4, са изпълнени. Системата е разположена в производствена среда и е достъпна за използване. Посоките за по-нататъшно развитие са очертани в глава 5.

Литература

1. React. Официална документация. <https://react.dev>
2. React Router. Официална документация. <https://reactrouter.com>
3. Vite. Официална документация. <https://vite.dev>
4. Node.js. Официална документация. <https://nodejs.org/docs>
5. Express. Официална документация. <https://expressjs.com>
6. Prisma. Официална документация. <https://www.prisma.io/docs>
7. PostgreSQL. Официална документация. <https://www.postgresql.org/docs>
8. Tailwind CSS. Официална документация. <https://tailwindcss.com/docs>
9. daisyUI. Официална документация. <https://daisyui.com>
10. Leaflet. Официална документация. <https://leafletjs.com>
11. OpenStreetMap. <https://www.openstreetmap.org>
12. Overpass API. OpenStreetMap Wiki. https://wiki.openstreetmap.org/wiki/Overpass_API
13. OpenRouteService. Документация на приложно-програмния интерфейс. <https://openrouteservice.org/dev/#/api-docs>
14. Open-Meteo. Документация на приложно-програмния интерфейс. <https://open-meteo.com/en/docs>
15. Stripe. Документация на приложно-програмния интерфейс. <https://docs.stripe.com/api>
16. Resend. Официална документация. <https://resend.com/docs>
17. Anthropic. Документация на приложно-програмния интерфейс. <https://docs.claude.com>
18. Jones, M., Bradley, J., Sakimura, N. RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force, 2015. <https://datatracker.ietf.org/doc/html/rfc7519>
19. OWASP Top Ten. Open Worldwide Application Security Project. <https://owasp.org/www-project-top-ten/>
20. OWASP. A01:2021 - Broken Access Control. https://owasp.org/Top10/A01_2021-Broken_Access_Control/

Всички електронни източници са посетени за последен път на 28.08.2026 г.

Приложения

Приложение 1. Изходен код на системата

Пълният изходен код на системата е публикуван в публично хранилище в платформата GitHub. Хранилището съдържа клиентската част, сървърната част, схемата на базата данни и скриптовете за първоначално зареждане на данни. Хранилището е достъпно на следния адрес:

<https://github.com/DenisHerzhyk/hiking-ecommerce/tree/fix/access-control>

В хранилището е приложен файл README.md, който описва структурата на проекта, необходимите променливи на средата и стъпките за локално стартиране на приложението.